LLMOps: API Cost, Latency and Prompt Version Management

MLOps - Assignment 2 – EduAssist AI

Student: **2025AE05909** | BITS ID: 2025AE05909 | Date: 5 November 2026

Repository: https://github.com/rajesh-302/https-github.com-rajesh-302-2025ae05909-rajesh-mlops-assignment-2

1. LLMOps vs MLOps

EduAssist AI needs an assistant that can answer academic questions and provide answers in a fixed JSON format. In MLOps, we mainly manage datasets, features and trained models. But in LLMOps, we also need to keep track of prompts, model or API versions and output rules because even small changes in these can affect the answers. The cost depends on the number of tokens used, while the response time depends on the prompt length, answer length and model provider. Since the answers may not always be the same, we also need automatic checks to make sure the output is valid JSON.

2. Setup

I used the Google Gemini API free tier through its OpenAI-compatible Chat Completions endpoint. I used the OpenAI Python library (version 3.45.0) with the gemini-3.5-flash-lite model. The temperature was set to 0 and the maximum output was limited to 400 tokens. I added a 5-second gap between API calls and allowed up to two retries if errors such as HTTP 429 or 503 occurred.

The API key was stored in the environment variable LLM_API_KEY inside a .env file, which was excluded from Git. I made sure that the key was never printed or stored in logs. Only general academic questions were sent to the API.

For cost calculation, I referred to Google's published paid-tier pricing on 5 November 2026. The rates were $0.30 per 1 million input tokens and $2.50 per 1 million output tokens, including thinking tokens.

For experiment tracking, I used MLflow version 3.18.0 with SQLite as the backend and created an experiment named eduassist_llmops_assignment.

3. Task 1 – Latency

Table 1: Five requests with prompt v1 (outputs/latency_tokens_cost.csv)

Question	Latency (s)	Status	Attempts	Input tok.	Output tok.	Total tok.	Cost (USD)
Q1 supervised vs unsupervised	5.616	success	1	21	396	417	0.000996
Q2 overfitting	1.802	success	3	24	245	269	0.000620
Q3 binary search	5.307	success	1	22	485	507	0.001219
Q4 Newton's second law	**8.299**	success (empty)	3	27	0	27	0.000008
Q5 mitochondria	1.572	success	3	22	234	256	0.000592

Latency statistics: mean 4.519 s, median 5.307 s, min 1.572 s, max 8.299 s, standard deviation 2.837 s.

Q4 was the slowest request, taking 8.30 seconds, which was 1.8 times the average time. It required three attempts and still returned an empty answer with zero output tokens, even though the HTTP status was 200. The same issue happened with Q4 when using prompt version v3. This suggests that the problem may be related to free-tier API limits or a content filter rather than the length of the response.

For the other requests, the response time was mostly related to the number of output tokens. For example, Q3 generated 485 tokens and took 5.3 seconds, while Q5 generated 234 tokens and took 1.6 seconds. The recorded latency includes only the successful attempt. However, the timestamps show gaps of 27–61 seconds between attempts, which means users may actually have to wait much longer when retries happen.

4. Tokens and Cost

I calculated the cost using the input and output token counts and the published API rates. To estimate the monthly cost, I multiplied the average cost per request by 10,000 requests.

Most of the cost (around 99%) comes from output tokens because they are more expensive than input tokens. In total, there were 1,360 output tokens compared to only 116 input tokens. Output tokens cost around 8.3 times more than input tokens.

The average cost was $0.000687 per request, which comes to around $6.87 per month. If we exclude the empty Q4 response, the average cost increases to $0.000857 per request, giving a monthly estimate of $8.57.

Recommendation: I would suggest keeping the responses short by setting a word limit and reducing the max_tokens value. If we reduce the output length by half, the estimated monthly cost will come down to around $3.47.

This was also observed in Task 3. The concise JSON prompt generated an average of 94 output tokens and had an estimated monthly cost of $2.61, which is 64% lower than prompt v1. Prompt caching may not provide much cost savings in this case because input tokens are already cheaper than output tokens.

5. Prompt Versioning and Evaluation

Table 2: Prompt comparison on the same five questions (outputs/prompt_evaluation_results.csv)

Prompt Version	Avg Latency (s)	Avg Input Tokens	Avg Output Tokens	Avg Cost (USD)	Valid JSON Rate	Recommended?
v1 – Basic	2.401	23.8	290.0	0.000732	0.0	No
v2 – Structured	6.788	47.2	213.2	0.000547	0.0	No
v3 – JSON Output	8.155	85.2	94.0	**0.000261**	**0.8**	**Yes**

Median latency: v1 2.33 s, v2 2.27 s, v3 6.16 s. Success rate: v1 0.8 (Q1 HTTP 503 after three attempts; v1 averages use 4 requests), v2 and v3 1.0. Schema-valid rate: 0, 0, 0.8. Keyword coverage: 0.80, 0.85, 0.80. Monthly cost (10k): USD 7.32, 5.47, 2.61.

I created three versions (v1, v2 and v3) of the eduassist-academic-qa prompt and maintained commit messages for each version. The differences between the prompts are saved in outputs/prompt_diff_*.txt. I ran each version separately in MLflow and used mlflow.genai.evaluate to check whether each response was valid JSON. The results were recorded as True or False and used to calculate the JSON pass rate. Version 3 was assigned the production alias, as shown in screenshots 01–05.

Why I selected v3 for production:

The main requirement of EduAssist is to return answers in a valid JSON format because the application needs to process them. Both v1 and v2 failed to produce valid JSON, with a pass rate of 0.0. Version 3 produced valid JSON for 4 out of 5 questions, giving it a pass rate of 0.8. The only failure was Q4, which returned an empty response. This was the same question that had an issue in Task 1, so it was not a JSON formatting error.

In terms of answer quality, all three versions performed similarly. The keyword coverage was 0.80 for v1, 0.85 for v2 and 0.80 for v3. However, v3 covered all the expected concepts in its four successful responses and also included a confidence level and key points.

Version 3 used more input tokens (85 compared to 24–47 for the other versions) because it had additional instructions for the JSON format. However, it generated shorter answers, averaging only 94 output tokens compared to 213–290 for v1 and v2. Because of this, v3 was also the cheapest, costing around $0.00026 per request or $2.61 per month. The estimated monthly cost for v1 and v2 was between $5.47 and $7.32.

One drawback of v3 was its response time. Its median latency was 6.2 seconds, compared to around 2.3 seconds for the other versions. This was partly due to retries caused by free-tier API limits, as three out of five v3 requests needed multiple attempts. Since EduAssist is an academic question-answering assistant

and does not require real-time responses, I consider correct formatting and lower cost more important than a few extra seconds.

Based on these results, I selected v3 as the production version. However, its JSON pass rate is only 0.8, so further improvements are needed. I recommend enabling Gemini's JSON response mode, adding retries for empty or invalid responses, and monitoring the JSON validity rate to make sure the assistant consistently returns the expected format.

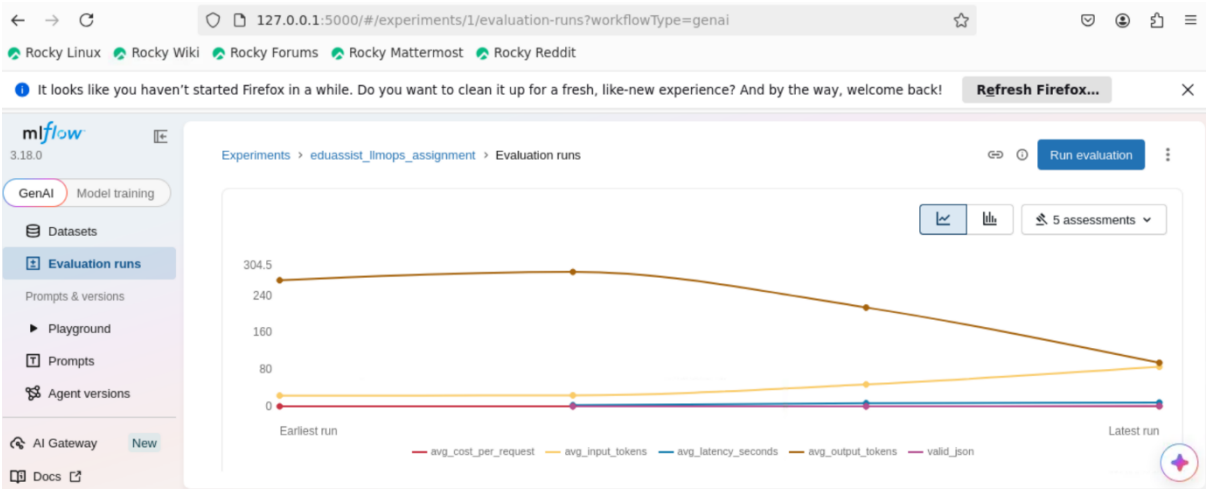
6. Conclusion and Limitations

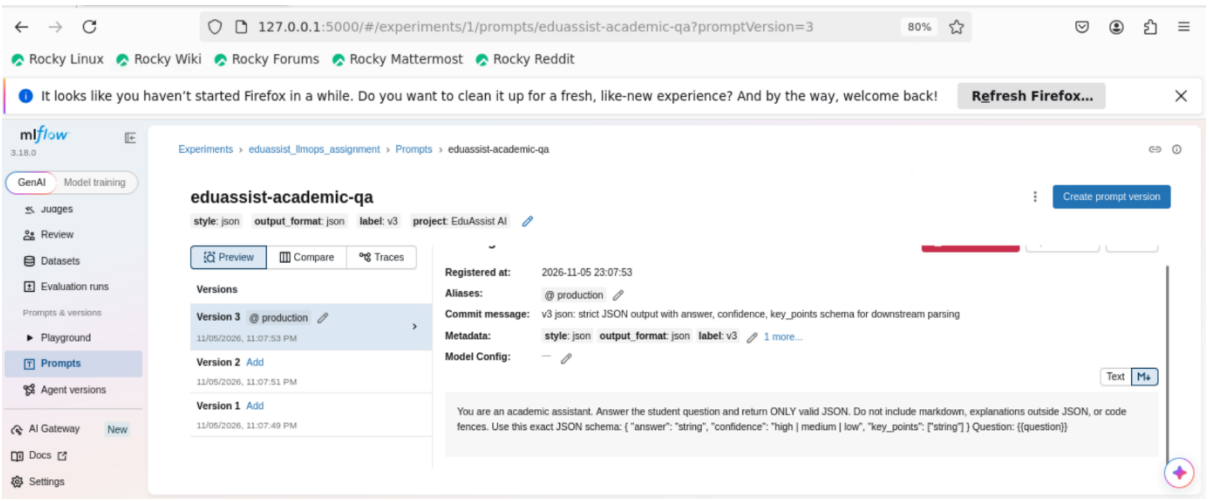
In this assignment, I measured the response time, token usage and cost for each request. I also created three prompt versions, tested them using MLflow and promoted v3 as the production version.

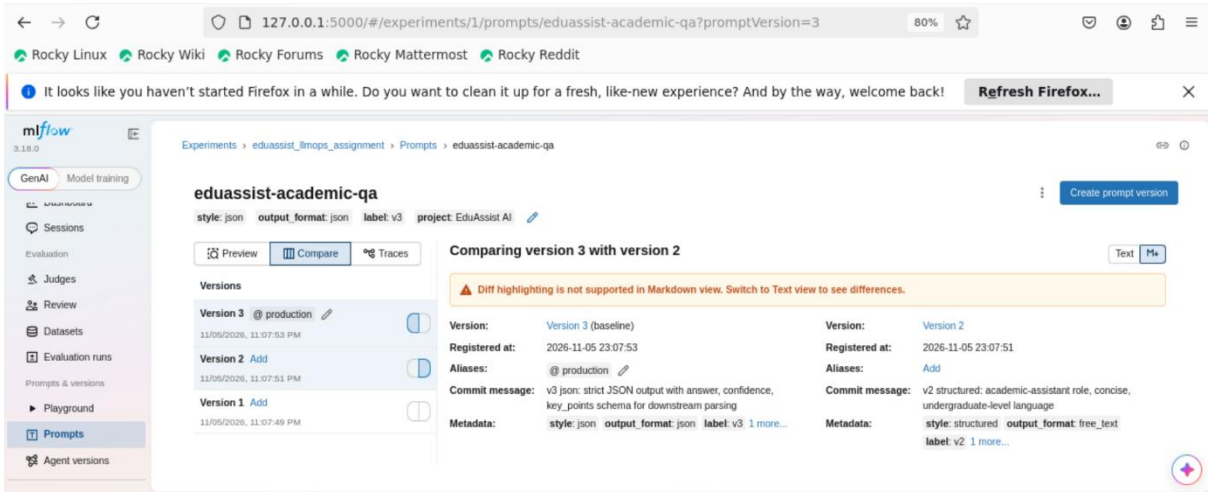
There are a few limitations in my testing. I used only five questions and ran each prompt version once, so the average results can be affected by unusual response times and API throttling. For example, Q4 in v2 took 25 seconds. I also did not record the finish reasons for the empty Q4 responses, so the exact reason for those failures is still unclear.

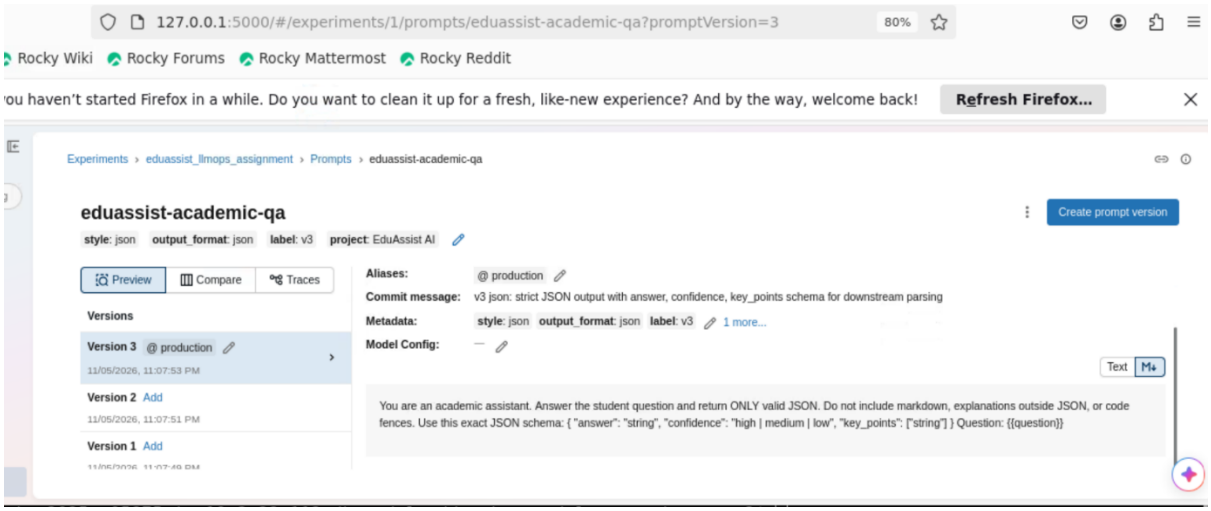
Another observation was that the completion tokens went beyond the configured max_tokens limit (485 tokens compared to 400). This may be because hidden thinking tokens are also included in the token count. Also, keyword coverage is only a basic way to measure answer quality and may not fully reflect how accurate or useful an answer is.

7. Experiment Screenshots






127.0.0.1:5000/#/experiments/1/prompts/eduassist-academic-qa?promptVersion=3
80%
Rocky Linux
Rocky Wiki
Rocky Forums
Rocky Mattermost
Rocky Reddit
It looks like you haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back!
Refresh Firefox...
mlflow
3.18.0
GenAI
Model training
Sessions
Evaluation
Judges
Review
Datasets
Evaluation runs
Prompts & versions
Playground
Prompts
Agent versions
Experiments › eduassist_llmops_assignment › Prompts › eduassist-academic-qa
eduassist-academic-qa
style: json
output_format: json
label: v3
project: EduAssist AI
Create prompt version
Preview
Compare
Traces
Versions
Version 3 @ production
11/05/2026, 11:07:53 PM
Version 2 Add
11/05/2026, 11:07:51 PM
Version 1 Add
11/05/2026, 11:07:49 PM
Comparing version 3 with version 2
Text
Diff highlighting is not supported in Markdown view. Switch to Text view to see differences.
Version: Version 3 (baseline)
Registered at: 2026-11-05 23:07:53
Aliases: @ production
Commit message: v3 json: strict JSON output with answer, confidence, key_points schema for downstream parsing
Metadata: style: json output_format: json label: v3 1 more...
Version: Version 2
Registered at: 2026-11-05 23:07:51
Aliases: Add
Commit message: v2 structured: academic-assistant role, concise, undergraduate-level language
Metadata: style: structured output_format: free_text label: v2 1 more...


127.0.0.1:5000/#/experiments/1/prompts/eduassist-academic-qa?promptVersion=3
80%
Rocky Wiki
Rocky Forums
Rocky Mattermost
Rocky Reddit
ou haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back!
Refresh Firefox...
Experiments › eduassist_llmops_assignment › Prompts › eduassist-academic-qa
eduassist-academic-qa
style: json
output_format: json
label: v3
project: EduAssist AI
Create prompt version
Preview
Compare
Traces
Versions
Version 3 @ production
11/05/2026, 11:07:53 PM
Version 2 Add
11/05/2026, 11:07:51 PM
Version 1 Add
Aliases: @ production
Commit message: v3 json: strict JSON output with answer, confidence, key_points schema for downstream parsing
Metadata: style: json output_format: json label: v3 1 more...
Model Config: —
Text
You are an academic assistant. Answer the student question and return ONLY valid JSON. Do not include markdown, explanations outside JSON, or code fences. Use this exact JSON schema: { "answer": "string", "confidence": "high | medium | low", "key_points": ["string"] } Question: {{question}}